

AI GCSE Revision Agent — Complete System Specification

1. Project Objective

Build a personal AI revision system that can:

1. Accept a GCSE textbook chapter or other legitimate study resource.
2. Identify the relevant subject, exam board, qualification and topic.
3. Find and verify the current official syllabus/specification.
4. Search the internet for relevant and reliable information when required.
5. Map textbook/resource content against the syllabus.
6. Identify what knowledge needs to be learned.
7. Generate exam-focused Anki flashcards.
8. Check generated cards for quality and accuracy.
9. Detect duplicates against the user's existing Anki collection.
10. Determine the correct Anki deck and note type.
11. Create and manage Anki decks and subdecks when requested.
12. Allow the user to approve generated cards and changes.
13. Add approved cards to Anki automatically.
14. Record what has been processed so the system can avoid unnecessary repetition.
15. Track syllabus coverage and identify knowledge gaps.
16. Eventually process new textbook chapters automatically.

The system should initially use **one main AI agent with several tools**, rather than immediately creating a complicated multi-agent architecture.

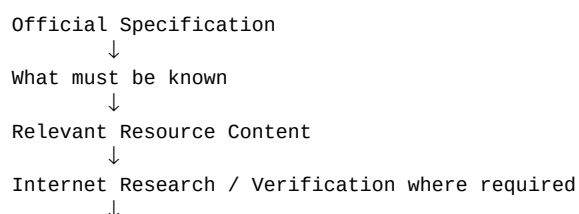
2. Core Principle

The central principle of the project is:

> The syllabus determines what needs to be learned; legitimate resources provide the information; internet research can supplement and verify that information; the AI turns the relevant information into high-quality cards; Anki stores and manages those cards.

The agent should **not simply summarise textbooks**.

Its job is to build a reliable chain:

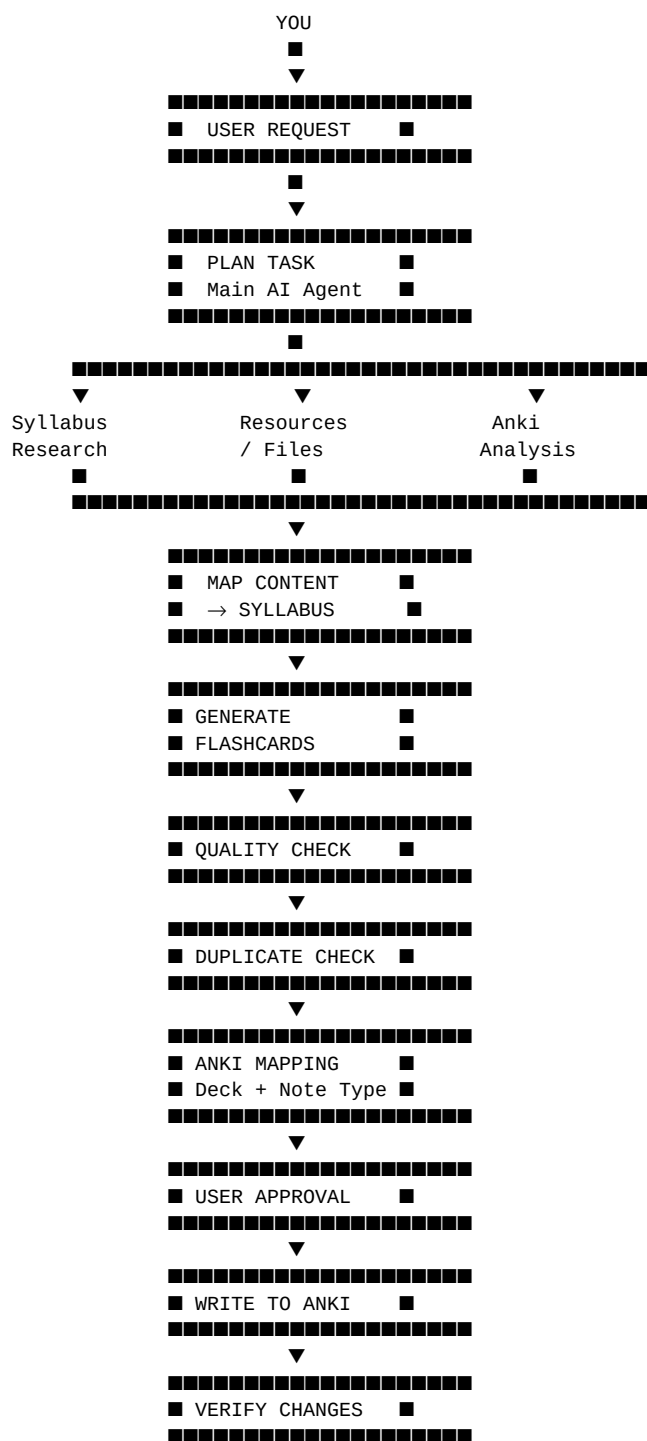


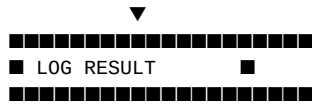
```

Flashcards
↓
Quality Control
↓
Duplicate Check
↓
Correct Anki Deck
↓
User Approval
↓
Anki

```

3. Overall Architecture





4. Main Agent

The system should initially have **one main AI agent**.

The agent is responsible for coordinating the workflow and deciding which tools it needs.

It should be able to:

- Understand the user's request.
- Identify task requirements.
- Create a processing plan.
- Search the internet.
- Search for syllabus information.
- Read resources.
- Map resources to syllabus requirements.
- Generate flashcards.
- Request quality checks.
- Check Anki.
- Determine the appropriate deck.
- Create decks/subdecks when requested.
- Determine the correct note type and fields.
- Request user approval.
- Add approved cards.
- Modify Anki content where explicitly requested.
- Verify changes.
- Record the processing result.

The LLM acts as the reasoning engine while the application provides the tools and controls.

5. Initial Agent Tools

The first version should provide tools similar to:

```
search_web()  
search_syllabus()  
read_file()  
read_pdf()
```

```
get_anki_decks()
create_anki_deck()
rename_anki_deck()

get_anki_note_types()
get_anki_fields()
find_anki_notes()
get_anki_note_info()

add_anki_cards()
update_anki_card()
move_anki_cards()
```

The exact tool implementation can evolve as the project develops.

6. User Request

The user should eventually be able to give a simple request such as:

> Process CAIE IGCSE Chemistry Topic 6 using Chapter 6 of my textbook.

The agent should determine:

```
Subject: Chemistry
Exam board: CAIE
Qualification: IGCSE
Topic: 6
Resource: Chapter 6
Target deck: GCSE::Chemistry
```

Where possible, this information should come from configuration rather than requiring the AI to guess.

Example:

```
{
  "chemistry": {
    "exam_board": "CAIE",
    "qualification": "IGCSE",
    "anki_deck": "GCSE::Chemistry"
  }
}
```

If information is missing, the agent should use available configuration, the user's request, resource metadata, and reliable research to determine it.

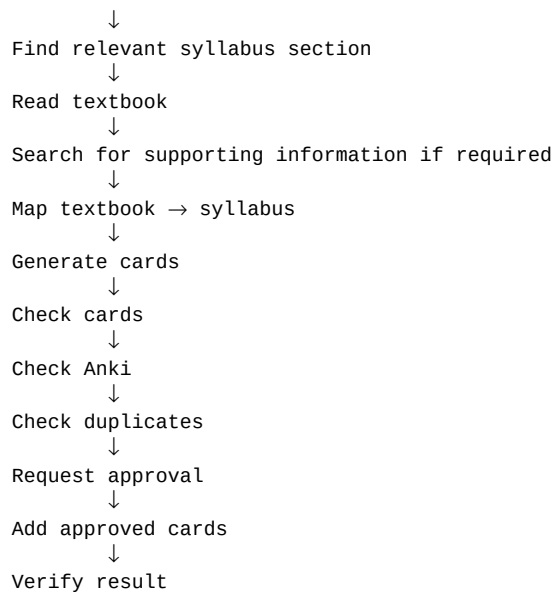
It should ask the user when ambiguity cannot be safely resolved.

7. Task Planning

Before performing a significant task, the agent should create a plan.

Example:

```
Identify syllabus
```



The interface should ideally show the user the current stage.

Example:

```
■ Processing Chemistry Topic 6  
  
✓ Identified exam board: CAIE  
✓ Found current syllabus  
✓ Found Topic 6  
✓ Read Chapter 6  
→ Mapping content to syllabus...  
→ Generating cards...
```

8. Syllabus Research

Syllabus research should happen **before flashcard generation**.

The system should prioritise authoritative sources, especially the official examination-board specification.

The agent should determine:

```
Exam board  
Qualification  
Specification version  
Relevant topic  
Learning objectives
```

The system should record:

```
Exam board: CAIE  
Qualification: IGCSE Chemistry  
Specification version: [version]  
Source: [source]  
Date accessed: [date]
```

The syllabus should be treated as the primary authority for determining what students need to know.

If the specification has changed, the agent should use the current relevant specification unless the user explicitly requests an older version.

9. Internet Research Capability

The agent/s should be able to **search the internet for relevant information**.

Internet research may be used to:

- Find the current official specification.
- Find relevant syllabus information.
- Verify information.
- Find authoritative explanations.
- Find legitimate educational resources.
- Fill gaps where the textbook does not sufficiently cover a syllabus requirement.
- Investigate unfamiliar concepts.
- Find current information when the task requires it.

The source priority should generally be:

```
Official examination board
↓
Official government/educational source
↓
Reputable educational source
↓
Other reliable source
```

The agent should not automatically treat every internet source as authoritative.

Where practical, research should record:

```
Source
URL
Date accessed
Information obtained
Purpose
```

Internet research should **supplement** the official specification rather than replace it.

The agent should distinguish between:

```
Official specification information
Textbook information
Internet research
AI inference
```

This helps maintain traceability and reduces the risk of unsupported information entering flashcards.

10. Resource Processing

The system should process the user's textbook or other legitimate study resources.

The preferred workflow is:

```
Textbook
↓
Extract text
↓
Identify chapter
↓
Split into sections
↓
Map sections against syllabus
```

The system should **not simply summarise the entire textbook**.

It should identify the content that corresponds to the syllabus requirements.

Example:

```
Textbook Section 6.3
↓
Syllabus Requirement 6.3.2
↓
Relevant
↓
Send to flashcard generation
```

11. PDF and Document Processing

The system should support common legitimate study-resource formats, particularly PDFs.

The preferred workflow is:

```
PDF
↓
Extract text
↓
Identify relevant pages
↓
Split into manageable sections/chunks
↓
Map against syllabus
↓
Generate cards
```

The system should eventually support:

- PDF text extraction.
- Chapter detection.
- Page detection.
- Section detection.
- Chunking.

- OCR for scanned documents.
- Handling context/token limits.
- Processing large textbooks without unnecessarily sending the entire document to the LLM.

For example:

```

500-page textbook
  ↓
Chapter 6
  ↓
Relevant pages
  ↓
Relevant sections
  ↓
AI

```

12. Syllabus Coverage Tracker

The system should maintain a coverage tracker.

Example:

| Syllabus point | Coverage | Cards |

|---|---|---:|

| 6.1 | ■ Complete | 8 |

| 6.2 | ■ Complete | 12 |

| 6.3 | ■■ Partial | 4 |

| 6.4 | ■ Missing | 0 |

Coverage should be based on the actual syllabus requirements.

Possible statuses:

```

Not Started
Partial
Complete

```

The agent should **not create cards simply because a topic exists in the syllabus**.

If a syllabus topic has not been started or the available resource contains no relevant information, the system should not generate cards for it merely to fill a perceived gap.

If the textbook does not sufficiently cover a syllabus requirement, the agent may use legitimate internet research or another legitimate resource to determine whether the missing information can be filled.

13. Flashcard Generation

Flashcards should be generated from the combination of:

```
Syllabus requirements
+
Relevant textbook/resource content
+
Verified supporting information where required
```

Cards should be structured rather than returned as arbitrary text.

Example:

```
{
  "front": "What is an isotope?",
  "back": "Atoms of the same element with the same number of protons but different numbers of neutrons.",
  "syllabus_point": "1.2",
  "topic": "Atomic structure",
  "tags": [
    "CAIE",
    "Chemistry",
    "AtomicStructure"
  ]
}
```

Every card should retain its relationship to the relevant syllabus point where possible.

14. Flashcard Generation Rules

The system should support configurable flashcard preferences.

Default preferences should include:

- ✓ Exam-focused
- ✓ Concise
- ✓ One useful idea per card
- ✓ Use GCSE terminology
- ✓ Avoid unnecessary information
- ✓ Avoid unnecessary examples
- ✓ No practice questions unless explicitly requested
- ✓ No duplicate concepts
- ✓ Cover relevant syllabus requirements

The system should prioritise **quality over quantity**.

It should not generate additional cards simply to increase the number of cards.

15. Flashcard Quality Requirements

The system should check generated cards for:

Accuracy

Is the information correct?

Syllabus relevance

Does the card relate to an actual syllabus requirement?

Completeness

Does the answer contain the required information?

Question quality

Is the question clear and unambiguous?

Atomicity

Does the card test one useful idea rather than several unrelated ideas?

Redundancy

Is another generated card essentially the same?

Exam usefulness

Would knowing the information help with the relevant specification?

Appropriate difficulty

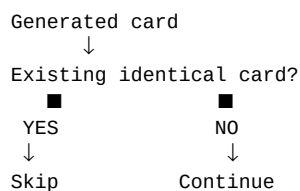
Does the card test an appropriate level of GCSE knowledge?

16. Duplicate Detection

Duplicate detection should have two stages.

Stage 1 — Exact Duplicate

Use normal programming to check whether an identical card already exists.



Stage 2 — Semantic Duplicate

Use AI to identify cards that have different wording but essentially test the same information.

Example:

Existing:

"What is the definition of an isotope?"

Generated:

"Define an isotope."

→ Semantic duplicate

The duplicate should normally be excluded.

The system should distinguish between:

Duplicate

vs

Related but useful

Related cards should not automatically be removed if they test genuinely different knowledge.

17. Anki Analysis

The system should inspect Anki before writing cards.

It should determine:

Available decks

Available subdecks

Available note types

Available fields

Existing notes

Existing cards

The system must **not assume** that a note type has fields such as:

Front

Back

FrontText

Instead, it should inspect the actual Anki note type and map the generated card to the fields that really exist.

This prevents template/field mismatches.

18. Correct Deck Selection

The agent should map subjects to the user's existing deck structure.

Example:

Chemistry

↓

GCSE::Chemistry

Biology

↓

GCSE::Biology

```
Physics
  ↓
GCSE::Physics

History
  ↓
GCSE::History
```

This should ideally be stored as configuration rather than guessed every time.

The user should also be able to override the default mapping.

19. Anki Deck and Subdeck Management

The agent/s should be able to **create and manage decks and subdecks when the user requests it**.

It should be able to:

- Create decks.
- Create subdecks.
- Check whether a deck already exists.
- Check whether a subdeck already exists.
- Avoid creating duplicate decks.
- Rename decks where supported and explicitly requested.
- Move cards between decks where supported and explicitly requested.
- Follow the user's requested hierarchy.
- Verify that requested deck changes were successful.

Example:

```
GCSE
  ■■■ Chemistry
    ■■■ Atomic Structure
```

The user should be able to request:

> Create an Atomic Structure subdeck inside GCSE::Chemistry.

The agent should inspect the existing structure first and then create the requested subdeck if it does not already exist.

The system must **not arbitrarily reorganise the user's Anki collection**.

Deck modifications should only occur when:

1. The user explicitly requests them, or
2. The user has previously authorised that behaviour through configuration.

20. Anki Toolset

The Anki integration should eventually support operations similar to:

```
get_anki_decks()
create_anki_deck()
rename_anki_deck()

get_anki_note_types()
get_anki_fields()

find_anki_notes()
get_anki_note_info()

add_anki_cards()
update_anki_card()
move_anki_cards()
```

The exact tool implementation can evolve depending on AnkiConnect's capabilities.

The agent should inspect the available Anki structure before making changes and verify the result afterward.

...

21. User Approval

The first versions should **not automatically add every generated card.**

Instead, show a review stage.

Example:

[illegible]

The user should be able to:

- Approve cards.
- Reject cards.
- Edit cards.
- Request regeneration.
- Approve individual cards.
- Approve groups of cards.
- Change the target deck.

- Request deck/subdeck creation.

Once the system is trusted, an automatic mode can be introduced for cards that pass all required checks.

22. Writing to Anki

After approval, the system should use AnkiConnect to add the cards.

The system should record the IDs of the notes it creates.

Example:

```
Processing ID: 8291
Anki Note ID: 17382947291
```

These IDs can later be used to identify or update cards.

After writing, the system should verify that:

- The cards were successfully created.
- They are in the correct deck.
- The correct note type was used.
- The expected fields were populated.

23. Processing Database

The system should maintain a record of previous processing.

Example:

Field Purpose
--- ---
File Resource processed
Subject GCSE subject
Exam board Exam board
Qualification Qualification
Syllabus version Specification used
Date processed Processing history
Cards generated Total generated
Cards added Successfully added
Duplicates Cards rejected as duplicates
Anki note IDs Created notes

| Target deck | Deck used |
| Coverage | Syllabus coverage |

This allows the system to know:

> "I've already processed this chapter."

It also allows the system to track which syllabus version and resources were used to create cards.

24. Processing History and Repeat Prevention

The system should prevent unnecessary repeated processing.

Before processing a resource, it should check:

```
Has this file been processed?  
Has this chapter been processed?  
Has this syllabus version changed?  
Have the relevant cards already been created?
```

If a resource has already been processed, the system should determine whether there is a genuine reason to process it again.

For example:

```
Same resource  
+  
Same syllabus version  
+  
Same processing configuration  
+  
Cards already exist  
↓  
Do not unnecessarily regenerate everything
```

If the syllabus has changed, the system may need to reassess the affected content.

25. Feedback Loop

The system should eventually support user feedback.

For example:

> These cards are too detailed.

The system can use that preference during future generation.

The project should maintain configurable flashcard preferences such as:

✓ Exam-focused

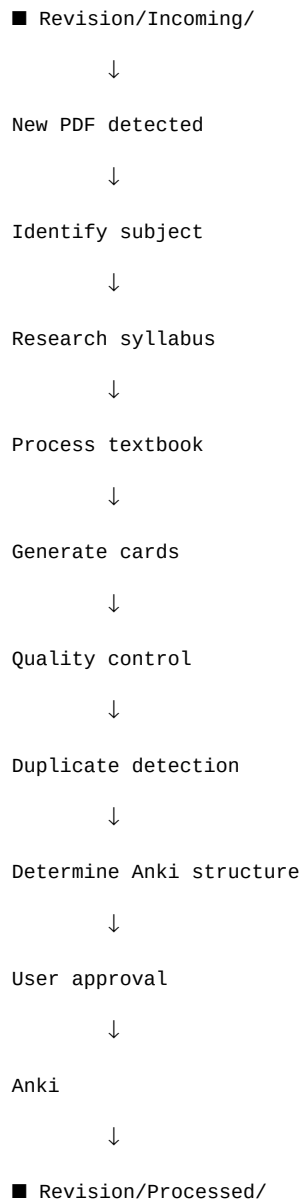
- ✓ Concise
- ✓ One fact per card
- ✓ Use GCSE terminology
- ✓ Avoid unnecessary examples
- ✓ No practice questions
- ✓ No duplicate concepts
- ✓ Cover every syllabus point

User preferences should be stored in configuration rather than being hard-coded into the agent wherever practical.

26. Automation

Once the manual workflow is reliable, the system can monitor a folder.

Example:

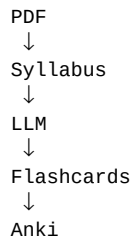


The eventual goal is:

> Put a legitimate textbook chapter into the folder → the agent processes it → reviewed/approved cards appear in the correct Anki deck.

27. Development Roadmap

V1 — Basic Pipeline



Features:

- Read PDF.
- Find syllabus.
- Generate structured cards.
- Connect to Anki.
- User approval.

V2 — Better Anki Integration

Add:

- Automatic deck mapping.
- Note-type detection.
- Field detection.
- Exact duplicate detection.
- Semantic duplicate detection.
- Deck/subdeck creation.
- Deck verification.

V3 — Syllabus Coverage

Add:

- Syllabus-point tracking.
- Coverage reports.

- Missing-content detection.
- Additional-resource research.
- Internet verification.
- Syllabus-version tracking.

V4 — Automation

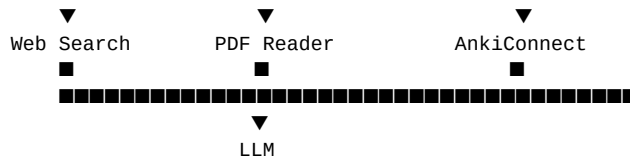
- Folder monitoring.
- Automatic processing.
- Processing history.
- Automatic duplicate prevention.
- Automatic resource classification.

V5 — Autonomous Revision System

```
graph TD; A[New resource] --> B[Agent identifies subject]; B --> C[Finds current specification]; C --> D[Researches relevant content]; D --> E[Maps content to syllabus]; E --> F[Generates cards]; F --> G[Checks cards]; G --> H[Checks Anki]; H --> I[Creates required deck/subdeck if authorised]; I --> J[Adds approved cards]; J --> K[Updates coverage]; K --> L[Records everything];
```

28. Initial Architecture

ONE MAIN AGENT

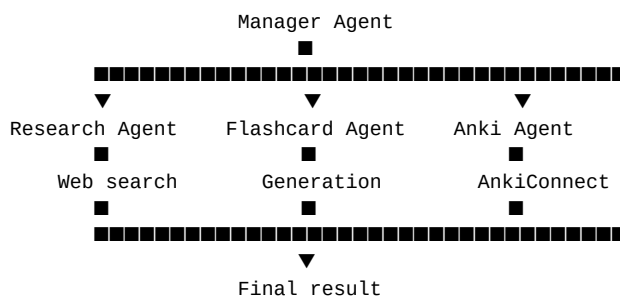


Do **not** immediately build a system of multiple agents.

Only introduce specialised agents when the complexity of the project makes it worthwhile.

29. Future Multi-Agent Architecture

If the system eventually becomes complex enough, responsibilities can be separated:



Possible roles:

Research Agent

Finds and verifies:

- Official syllabus.
- Syllabus version.
- Relevant syllabus points.
- Supporting information.
- Reliable educational resources.

Flashcard Agent

Creates:

- Questions.
- Answers.
- Tags.
- Topic information.
- Syllabus relationships.

Examiner/Quality Agent

Checks:

- Accuracy.
- Syllabus relevance.
- Completeness.
- Exam usefulness.
- Missing information.
- Redundancy.
- Duplicate concepts.

Anki Agent

Handles:

- Decks.
- Subdecks.
- Note types.
- Fields.
- Existing cards.
- Duplicate checking.
- Adding cards.
- Updating cards.
- Moving cards where authorised.

Manager Agent

Coordinates the other components and maintains the overall workflow.

This should be treated as a **future architecture**, not the starting point.

30. Safety and Change Control

The system should distinguish between **read actions** and **write actions**.

Read actions

Examples:

```
Search the internet
Read a PDF
Read the syllabus
Read Anki decks
Read existing Anki cards
```

Write actions

Examples:

- Create Anki cards
- Edit Anki cards
- Create decks
- Create subdecks
- Rename decks
- Move cards
- Delete or otherwise remove content

The system should require appropriate user approval for significant write actions, especially destructive changes.

The agent should never delete or substantially reorganise the user's existing Anki collection without explicit authorisation.

31. Configuration

Important user-specific settings should be stored in configuration.

Example:

```
{
  "subjects": {
    "chemistry": {
      "exam_board": "CAIE",
      "qualification": "IGCSE",
      "anki_deck": "GCSE::Chemistry"
    },
    "biology": {
      "exam_board": "[configured]",
      "qualification": "[configured]",
      "anki_deck": "GCSE::Biology"
    }
  }
}
```

Configuration may eventually contain:

- Exam boards
- Qualifications
- Subject mappings
- Deck mappings
- Flashcard preferences
- Syllabus versions
- Automatic-processing settings
- Approval settings

The agent should prefer configuration over guessing.

32. Core Data Flow

The complete processing pipeline should be:

```
USER REQUEST
↓
TASK PLANNING
```

```

↓
IDENTIFY SUBJECT / EXAM BOARD / QUALIFICATION
↓
FIND CURRENT OFFICIAL SPECIFICATION
↓
SEARCH INTERNET IF REQUIRED
↓
READ LEGITIMATE RESOURCE
↓
IDENTIFY RELEVANT CONTENT
↓
MAP RESOURCE → SYLLABUS
↓
IDENTIFY COVERAGE / GAPS
↓
GENERATE FLASHCARDS
↓
QUALITY CONTROL
↓
EXACT DUPLICATE CHECK
↓
SEMANTIC DUPLICATE CHECK
↓
INSPECT ANKI
↓
DETERMINE DECK / SUBDECK
↓
CREATE DECK / SUBDECK IF USER REQUESTS OR
AUTHORISED CONFIGURATION ALLOWS IT
↓
MAP NOTE TYPE + FIELDS
↓
USER REVIEW / APPROVAL
↓
WRITE TO ANKI
↓
VERIFY ANKI CHANGES
↓
UPDATE COVERAGE
↓
RECORD PROCESSING HISTORY

```

33. Project Success Criteria

The system should eventually be considered successful when the user can do something as simple as:

> **Process Chapter 6 of my CAIE IGCSE Chemistry textbook.**

and the system can reliably:

1. Identify Chemistry.
2. Identify CAIE IGCSE Chemistry.
3. Find the current official specification.
4. Identify the relevant Topic 6 requirements.
5. Read the requested textbook chapter.
6. Map the chapter to the specification.
7. Identify what needs to be learned.
8. Search the internet if legitimate supporting information is required.

9. Generate high-quality exam-focused cards.
10. Check cards for accuracy and relevance.
11. Detect exact and semantic duplicates.
12. Inspect the user's Anki collection.
13. Identify or create the correct deck/subdeck as authorised.
14. Detect the correct note type and fields.
15. Show the cards for approval.
16. Add approved cards to Anki.
17. Verify that they were added correctly.
18. Update the syllabus coverage tracker.
19. Record what was processed.
20. Avoid unnecessary duplication in future processing.

34. Final System Definition

The finished project should be:

> A personal AI-powered GCSE revision agent that uses official specifications as the source of truth, processes legitimate study resources, searches the internet for relevant and reliable supporting information, maps knowledge to syllabus requirements, generates and quality-checks exam-focused Anki flashcards, detects duplicates, manages Anki decks and subdecks according to the user's instructions, obtains approval for appropriate changes, writes approved content to Anki, tracks syllabus coverage and processing history, and eventually automates the entire workflow from new study resource to organised revision cards.

The project should begin as a **single agent with tools** and evolve into a multi-agent architecture only if the increasing complexity genuinely justifies it.